Day 52 – API Management (APIM) – Part 1

Class date: 9th September 2026 | Module: SAP API Management / Integration Suite

---

Good accuracy today. Your terminology for **Developer Hub** as the centralized place where all APIs are managed is exactly SAP's own official term (confirmed against SAP Help documentation) – many learners assume it's informal shorthand, but it's the real product name. Everything else below checked out too; I've just organized it and added a bit more precision in a couple of spots.

1. Quick recap from Day 51, before starting APIM

JMS Adapter = guarantees message delivery, via its Retry Mechanism. **Very important interview question:** *"How do you implement the Retry Mechanism in SAP CPI?"* → **Answer: use the JMS Adapter.**

2. What is API Management? (start here first)

If you want to **create, secure,** and **manage** any APIs, you use **API Management.**

What is an API? For example: if you want to connect to any system, you use an API to do it. Different ways exist to connect systems – **SFTP, HTTP, Queues** – and **API** is one more of these connection methods.

An **Endpoint URL** is what an API exposes to be called. Common API types: **SOAP API, REST API, OData API.**

Terminology: if you're connecting using an API, that connection is referred to as an **API-based connect.** This is distinct from a **File-based Interface** or a **Queue-based Interface** – when you're connecting using API-based connects, it behaves like a **real-time integration.**

3. Near Real-Time vs. Real-Time Integration

Scenario: suppose a file is placed on a Target system via **Scheduling.** Source → SFTP → file placed → a **Scheduled time** is what actually picks up the file.

This is Near Real-Time, not true real-time – it's nothing but a scheduled time at which the files get picked up (e.g. every 10 minutes, every hour). **True Real-Time** integration, by contrast, is achieved through **API Management.**

If you want to go with genuinely real-time integrations, you use the API Management service.

Legacy systems, historically: true Real-Time integration was **not commonly available** (X); what was typically used instead was **Near-Real-Time integration via Scheduling** (✓) – i.e. file/batch-based interfaces running on a timer, rather than instant API-driven calls.

Example – old-time money transfer: in the past, transferring money meant physically **going to the bank** and making a **cash deposit request**, which was then **manually deposited** by bank staff – a slow, non-real-time, manual process.

4. Now, the Digital Platform era

Example: UPI transactions – GPay, PhonePe, etc. – these are true real-time, API-driven transactions.

Example – Amazon "My Orders": when you open the Orders Information / "My Orders" page, a **GET** call fetches **all orders** from the **Backend System**, and that data is then **fetched and displayed** on the **Frontend.** The Backend System here would be something like **S/4HANA (the SAP ERP system).**

Representation, as drawn in class:

S/4HANA exposes multiple APIs for different business objects – e.g. **Business Partner**, **Business Partner Address**, **Contact Details**, **Vendors/Orders**. A **Client** sends a **Request** to the Backend URL of S/4HANA (the **Gateway System**, running **On-Premise**, holding the **Master Data**), and S/4HANA sends back a **Response.**

```
Client -> Request (Backend URL) -> S/4HANA (Gateway System, On-Premise, Master Data)
Client <- Response <- S/4HANA
```

5. API Gateway & Load Balancer

Gateway System → **API Management.** A **Load Balancer** processes requests **sequentially**, distributing load across backend instances.

Example: 10,000 calls hitting S/4HANA are distributed through a **Load Balancer**, sitting behind the **API Gateway / API Proxy** layer of API Management:

```
Client App -> Request (Proxy URL) -> API Gateway / API Proxy (API Management) ->
Load Balancer -> S/4HANA Client App <- Response <- API Gateway / API Proxy <-
S/4HANA
```

The URL the Client App actually calls is the URL of API Management (the Proxy URL) – not S/4HANA's real backend URL directly.

Why this matters – S/4HANA has many different APIs. To manage all of them centrally, that centralized hub is called the **Developer Hub** – this is where **all the APIs are managed.**

6. API Gateway – the core idea

API Gateway: instead of giving our real backend URL to a third-party system, we give them the **API Gateway Proxy URL** instead.

Analogy used in class: imagine one area with 2 houses – which house do you want to secure? **Your own house.** In the same way, in real-time integration scenarios (the Digital Platform world), **your own internal systems get secured first**, instead of relying on other/external systems to secure themselves.

Pass-through scenario:

```
3rd Party System -> API Management -> S/4HANA (and the reverse direction too)
S/4HANA -> CPI -> 3rd Party
```

Pass-through scenario means: you are simply getting data from S/4HANA and sending that data **as-is** – for this, you use **API Management** (no transformation involved).

If any transformation is needed, then **CPI is used in between:**

```
3rd Party System -> API Management (Routing) -> CPI (Transformation) -> S/4HANA
```

Key distinction: API Management handles **routing** (and security); **CPI handles transformation/mapping. API Management is NOT used for mapping or transformation** – that's squarely CPI's job.

7. Capabilities only API Management has (not CPI)

Question raised in class: suppose you need to **restrict access by IP address** – can this be done in CPI? **"No."** This can be done in **API Management** instead (via its Access Control / IP allow-list type policies).

Second example: suppose you need to accept only **100 requests in a single day** (rate limiting). **We cannot do this in CPI** – but **we can do it in API Management**, using its **Quota Policy** (rate limiting based on number of calls in a given time range – e.g. per day, per month) and **Spike Arrest Policy** (protects against sudden bursts/spikes of traffic).

Why we exactly use API Management: Secure, Manage, Routing.

API Management is an optional approach – not every integration scenario requires it; it's layered in specifically when you need governance capabilities (security, rate limiting, IP restriction, centralized API exposure) that CPI alone doesn't provide.

Another advantage: you can create **documentation for an API Proxy** directly within API Management – useful for consumers who need to understand how to call your APIs.

Example: assume an API call for **Orders (S/4HANA).** Just like **CPI has many monitoring options** (e.g. **Message Monitoring/Logs**, to get any message's logs), **API Management has its own equivalent monitoring/analytics capability** for tracking API traffic, errors, and usage.

Quick-recall summary

- **API Management** = create, secure, manage APIs; used for genuinely **real-time** integrations (vs. scheduled/near-real-time SFTP-style file integrations).
- **API Gateway/Proxy**: expose a Proxy URL to third parties instead of your real backend URL – protects and centralizes access to internal systems.
- **Developer Hub** = the official SAP term for the centralized place where all APIs are managed/published/subscribed to.
- **Pass-through scenario** (API Management only, data as-is) vs. needing **CPI in between** for transformation/mapping.
- Things CPI **cannot** do natively but API Management **can**: **IP restriction** and **rate limiting/Quota** (e.g. 100 requests/day), via Quota and Spike Arrest policies.
- API Management is an **optional** layer, used specifically for Secure / Manage / Routing capabilities, plus API documentation and its own analytics/monitoring.